

# Technical Documentation

Automated Deployment Pipeline & Observability (v3.0)

## 1. Architecture & Network Topology

This environment utilizes Infrastructure as Code (IaC) to automatically provision a secure, containerized, two-tier network architecture that separates public-facing traffic routing from backend application processing and database tracking.

- **Host Machine:** Developer Workstation.
- **Hypervisor:** VirtualBox (Managed via Vagrant).
- **Node 1 (Gateway):** Ubuntu 22.04 LTS | IP: `192.168.50.10` | Service: Nginx Reverse Proxy securely routing HTTP and WebSocket traffic to the hidden App Server.
- **Node 2 (App Server):** Ubuntu 22.04 LTS | IP: `192.168.50.11` | Services: Docker, Streamlit Application, SQLite, and a complete Observability Stack (Prometheus, Node Exporter, Grafana).

## 2. The Application Payload: doms-recon

The core workload is a custom Python web application ( `doms-recon` ) containerized via Docker. It serves as an automated incident reconciliation and filtering tool.

- **Strict Filtering Engine:** The application scans the Summary, Description, and Resolution Note columns, strictly isolating records containing "DOMS" or "Pump" while instantly dropping any record referencing "RFID".
- **Data Transformation:** Automatically injects required structural columns (Cause Category, Cause Code) and validates ticket creation dates against DOMS deployment schedules.
- **Enterprise UI/UX:** Served via Streamlit on port `8501`, featuring a responsive sidebar for data input, a live dataframe preview of processed rows, and dynamic Excel download generation.
- **Persistent Database Storage:** Integrates a serverless SQLite database ( `doms_history.db` ) mapped securely to the host VM via Docker Volumes ( `-v /opt/doms-recon:/app` ). Every processed batch is tracked by timestamp, and the output Excel files are stored directly inside the database as BLOBs to allow for historical downloads.

### 3. Observability & Monitoring Stack

---

A complete hardware and business intelligence monitoring trinity is deployed as a suite of Docker containers running alongside the Python application.

- **Node Exporter (Hardware Agent):** Runs on port `9100` to measure host CPU, RAM, and disk utilization. The container is specifically configured with `--net="host"`, `--pid="host"`, and `-v "/:/host:ro,rslave"` to ensure it monitors the actual Ubuntu host operating system rather than the container's own isolated filesystem.
- **Prometheus (Time-Series Database):** Runs on port `9090` and acts as the central scraper. It reads from a dynamically generated `prometheus.yml` configuration file to pull metrics from the Node Exporter every 15 seconds.
- **Grafana (Visualization Engine):** Runs on port `3000` (forwarded to the Windows host). It visualizes system health using the open-source "Node Exporter Full" dashboard (ID `1860`), backed by persistent storage ( `-v grafana-storage:/var/lib/grafana` ).
- **Business Intelligence Integration:** Grafana is equipped with the `frser-sqlite-datasource` plugin. The application's database folder is securely mounted as read-only ( `-v /opt/doms-recon:/app-data:ro` ), allowing Grafana to read the Streamlit application's internal database. Custom SQL queries are executed against the `run_history` table to visualize real-time DOMS ticket validation KPI counts natively alongside server health metrics.

### 4. Automated Provisioning (IaC)

---

The environment is entirely built using automated Bash scripts triggered by Vagrant, eliminating the need for manual configuration.

- **Gateway Provisioning (setup-gateway.sh):** Installs Nginx and configures a reverse proxy block with explicit `Upgrade` headers to maintain the persistent WebSocket connections required by Streamlit.
- **App Server Provisioning (setup-app.sh):** Installs Docker, creates dedicated monitoring directories, dynamically generates the `prometheus.yml` file, boots the application, and deploys the entire monitoring stack with strict volume mapping applied.

## 5. Troubleshooting Ledger

| Error / Symptom                                  | Root Cause                                                                                                                                                    | Resolution                                                                                                                                                   |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Blank Gray Screen &amp; WebSocket Failure</b> | Streamlit's CORS and XSRF security protocols actively blocked the connection because the traffic originated from a proxy.                                     | Appended --<br><code>server.enableCORS=false</code> and --<br><code>server.enableXsrfProtection=false</code><br>to the Docker run command.                   |
| <b>Persistent WebSocket Disconnect</b>           | Nginx configuration included a trailing slash, stripping the critical Streamlit <code>/_stcore/</code> stream routing.                                        | Implemented a dedicated location <code>^~ /_stcore/stream</code> block in Nginx with explicit HTTP Upgrade headers.                                          |
| <b>Data Loss on Reboot</b>                       | Docker containers are inherently ephemeral; destroying the container wiped the internal SQLite database.                                                      | Implemented a Docker Volume in the Bash script ( <code>-v /opt/doms-recon:/app</code> ) to permanently map the container's working directory to the host VM. |
| <b>ERR_CONNECTION_REFUSED on Port 3000</b>       | Adding a new port forward to the Vagrantfile requires a full network adapter reset. Running <code>`vagrant provision`</code> only updates internal software.  | Executed <code>vagrant reload appserver</code> to cleanly reboot the VM and successfully apply the new host-to-guest firewall rules.                         |
| <b>Flat or Empty Grafana Graphs</b>              | Grafana's default time range is set to the last 24 hours, making newly collected telemetry data (minutes old) appear microscopically compressed or invisible. | Adjusted the time picker in the top right of the dashboard from "Last 24 hours" to "Last 5 minutes" to zoom in on live data.                                 |

| Error / Symptom                            | Root Cause                                                                                                                                                                                                                                                                                              | Resolution                                                                                                                                                                                                                                         |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "No file exists at path" in Grafana SQLite | <ol style="list-style-type: none"><li>1. Python used a relative path, bypassing the shared volume.</li><li>2. The .db file wasn't generated until the web app's first run.</li><li>3. Strict Linux root permissions blocked Grafana's unprivileged user (UID 472) from reading the directory.</li></ol> | Updated the Python script to use absolute pathing (/app/doms_history.db), loaded the Streamlit web UI to trigger initial database generation, and ran <code>sudo chmod 777 /opt/doms-recon/doms_history.db</code> to open read-access for Grafana. |